

Enumerations and Nested Classes

Enumerations

An **enumeration** (enum) is a data type that contains a fixed set of constants. Enums are best used when all possibilities of the values or instances of the class are known. They can be declared on their own or within another class.

With numeric or String constants, an invalid value can pass and not be found until runtime. With enums, it is impossible to create an invalid enum value without introducing a compiler error.

Enumerations show up whenever a set of items whose types are known at compile time. Common examples include the following:

- Compass directions
- Months of the year
- Cards in a deck

To create an enum, declare a type with the enum keyword, an identifier of the kind, and a list of values called **enum constants**. The enum constants are the only allowed values for the type.

```
public enum Period {
    PRELIM, MIDTERM, PREFINAL, FINAL;
}
```

After creating an enumerated data type, declare variables of that type just like the following: **Period p;**

Assign any of the enum constants to the variable, such as the following statement: **p = Period.MIDTERM;**

An enumeration type such as *Period* is a class. Its enum constants act as objects instantiated from the class, including having access to the methods of the class. These built-in methods are non-static.

Method	Description	Example if p = Period.MIDTERM;
<code>toString()</code>	Returns the name of the calling constant object	<code>p.toString()</code> is "MIDTERM"
<code>ordinal()</code>	Returns an integer that represents the constant's position in the list of constants; the first position is 0	<code>p.ordinal()</code> is 1
<code>equals()</code>	Returns true if its argument is equal to the calling object's value	<code>p.equals(Period.PRELIM)</code> is false
<code>compareTo()</code>	Returns a negative integer if the calling object's ordinal value is less than that of the argument, 0 if they are the same, and a positive integer if the calling object's ordinal value is greater than that of the argument	<code>p.compareTo(Period.FINAL)</code> is negative <code>p.compareTo(Period.PRELIM)</code> is positive <code>p.compareTo(Period.MIDTERM)</code> is 0

Comparison operators can be used with enumeration constants instead of using the `compareTo()` method to return a number. Several static methods can also be used with enumerations. These are used with the type and not with the individual constants.

Below are two (2) useful static methods to use with enumerations.

Method	Description	Example with Period Enumeration
<code>valueOf()</code>	Accepts a string parameter and returns an enumeration constant	<code>Period.valueOf("PRELIM")</code> returns the <code>PRELIM</code> enum constant
<code>values()</code>	Returns an array of the enumerated constants	<code>Period.values()</code> returns an array with 4 elements that contain the enum constants

An enumerated type in its own file can be declared, where the filename matches the type name and has a `.java` extension. An enumerated type within a class but not within a method can also be declared. In the following example, a `Period` enumeration is declared, while a `Period` variable is declared in the main method. Then, an enhanced for loop declares a local `Period` variable named `p` that takes on the value of each element in the `Period.values()` array in turn so it can be displayed.

```
import java.util.Scanner;
public class EnumDemo {
    enum Period {PRELIM, MIDTERM, PREFINAL, FINAL};

    public static void main(String[] args) {
        Period pd;
        String input;
        int position;
        int comparison;
        Scanner sc = new Scanner(System.in);
        System.out.println("The grading periods are:");
        for(Period p : Period.values())
            System.out.print(p + " ");
        System.out.print("\nSelect a grading period: ");
        input = sc.nextLine().toUpperCase();
        pd = Period.valueOf(input);
        System.out.println("You entered " + pd);
        position = pd.ordinal();
        System.out.println(pd + " is in position " + position);
        System.out.println("So its period number is " + (position + 1));
    }
}
```

The user is prompted to select a period, which is converted to its uppercase equivalent. The `valueOf()` method is used to convert the user's input to an enumeration value. The program also retrieves the position of the selected period in the enumeration list.

Below is a sample output of the above code.

```
The grading periods are:
PRELIM MIDTERM PREFINAL FINAL
Select a grading period: Midterm
You entered MIDTERM
MIDTERM is in position 1
So its period number is 2
```

Enumerations can also be used to control a switch structure. The following example contains a class that declares an enumeration for pizza sizes and then uses a switch structure to display the prices.

```
import java.util.Scanner;
public class EnumDemo2 {
    enum Size {REGULAR, LARGE, PARTY};

    public static void main(String[] args) {
        Size s = Size.LARGE;
        System.out.print("Price of pizza is ");
        switch(s) {
            case REGULAR:
                System.out.println("381.00");
                break;
            case LARGE:
                System.out.println("602.00");
                break;
            case PARTY:
                System.out.println("799.00");
                break;
        }
    }
}
```

In the above program, s is assigned the LARGE constant, so the output is: Price of pizza is 602.00

Using enums makes the values type-safe; **type-safe** is used to describe a data type that allows only appropriate behaviors. As with other classes, methods can also be added and other fields to an enum type.

Nested Classes

In Java, it is possible to create a class within another class and store them together; such classes are called **nested classes**. The containing class is the top-level class. There are four (4) types of nested classes:

- **static member class** – has access to all static methods of the top-level class
- **Non-static member class (inner class)** – requires an instance; it has access to all data and methods of the top-level class.
- **Local class** – defined within a method body
- **Anonymous class** – a special case of a local class that has no identifier

The most common reason for nesting a class inside another is because the inner class is used only by the top-level class. Being able to package the classes together makes their connection easier to understand and their code easier to maintain.

The sample class below describes houses that are being sold by a real estate company. The class might contain separate fields for the listing number, price, address, and area. Although the listing number and price go with the real estate listing, decide if the address and area go with the house.

```
public class RealEstateListing {
    private int listingNumber;
    private double price;
    private HouseData hData;

    public RealEstateListing(int num, double price, String address, int sqFt) {
        listingNumber = num;
        this.price = price;
        hData = new HouseData(address, sqFt);
    }
    public void display() {
        System.out.println("Listing number #" + listingNumber + " Selling for Php" +
price);
        System.out.println("Address: " + hData.streetAddress);
        System.out.println(hData.squareFeet + " square feet");
    }
    private class HouseData {
        private String streetAddress;
        private int squareFeet;

        public HouseData(String address, int sqFt) {
            streetAddress = address;
            squareFeet = sqFt;
        }
    }
}
```

In the sample program, the inner *HouseData* class is a private class. An inner class is not necessary to make private, but doing so keeps its members hidden from outside classes. If class members must be accessible, do not make it an inner class. An inner class can access its top-level class's fields and methods, even if they are private, and an outer class can access its inner class's members.

Inner classes are not usually created. For example, if the *HouseData* class is made as a regular class (as opposed to an inner class) and stored in its own file, it can be used with composition in other classes. As it stands, it is usable only in the class in which it now resides. Nested classes can also not be created frequently, but they will be implemented in some built-in Java classes.

References:

Farrell, J. (2019). *Java programming*. Cengage Learning, Inc.
Oracle Academy (n.d.). Citing sources. Retrieved from <https://academy.oracle.com/>
Oracle Docs (n.d.). Citing sources. Retrieved from <https://docs.oracle.com/javase/tutorial/java/javaOO/index.html>